

ESL Design and Multi-Core Validation using the System-on-Chip Environment

Tarea 1

Gabriel Abarca
Jesús Alberto Castro
Fabio Jaramillo

Moisés Leiva
Noel Pérez

Tecnológico de Costa Rica
Diseño Alto Nivel, Prof: Luis G. León Vega, Ph.D
11 de junio 2026

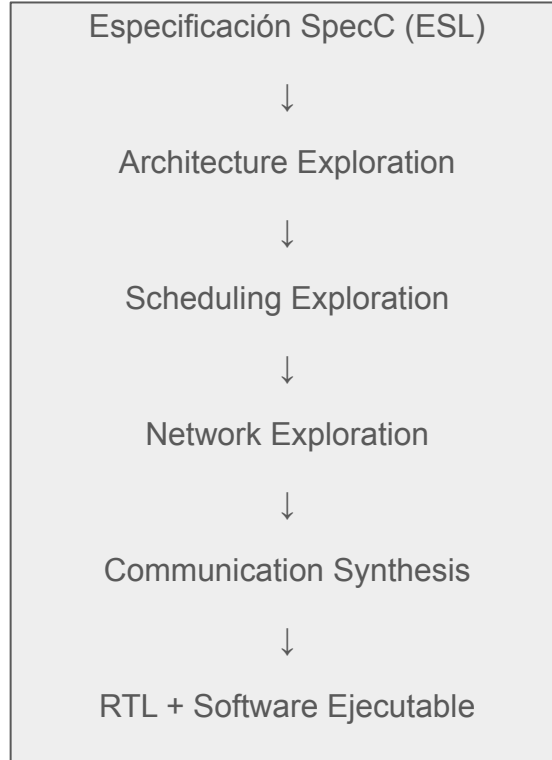
Problema principal

¿Cómo diseñar y validar **MPSoCs** complejos eficientemente y acelerar su simulación usando multi-core?

- **Desafío 1:** La creciente **complejidad** del diseño de sistemas embebidos.
- **Desafío 2:** La lentitud de la **simulación** tradicional.

Tipo de enfoque

Top-Down



Arquitectura del Sistema: Componentes principales

Elementos de Cómputo (PEs)

- Procesadores de Software embebidos (CPUs de propósito general y DSPs).
- Aceleradores de hardware dedicados (ASICs y bloques de Propiedad Intelectual / IP)

Infraestructura de Comunicación

- Redes de interconexión heterogéneas basadas en buses y puentes (bridges).
- Lógica automatizada de direccionamiento y sincronización.

Base de datos de Soporte (Modelos)

- Repositorio integrado con componentes de hardware, buses, periféricos y modelos de RTOS

Organización del sistema: Validación Paralela

El Planificador

- **Simulación Paralela:** El motor asigna múltiples hilos del sistema operativo en paralelo según los núcleos del anfitrión (host).
- **Optimización:** Rompe la limitación tradicional de los simuladores monohilo de eventos discretos.

Sincronización Segura

- **Candado central (L):** Protege los recursos globales del planificador y las colas de eventos.
- **Candados por canal (ch→L):** Cada canal actúa como un monitor para evitar colisiones de hilos concurrentes.

Restricciones de diseño

RESTRICCIONES DE DISEÑO EN MPSoC

Límites y requisitos que el diseño debe cumplir.

#	RESTRICCIÓN	¿QUÉ IMPLICA?	EJEMPLO / MÉTRICA
1	 Rendimiento (Performance)	- El sistema debe completar su ejecución dentro de un tiempo máximo. - Determina el throughput mínimo requerido.	- Decodificador H.264: 30 FPS (33.3 ms por frame) - Latencia máxima aceptable: $< X$ ms
2	 Tiempo Real (Real-Time)	- Las tareas críticas deben cumplir sus deadlines. - Importa la peor latencia (worst case).	- Deadline de tarea crítica = 10 ms - Jitter máximo permitido = 1 ms
3	 Área de Silicio (Area)	El diseño debe caber dentro del área máxima disponible del chip.	- Área disponible: 100 mm² - Área del diseño: ≤ 100 mm²
4	 Potencia (Power)	- El consumo total debe estar por debajo del presupuesto definido. - Considera potencia dinámica y estática.	- Presupuesto de potencia: 3 W - Consumo del diseño: ≤ 3 W
5	 Memoria (Memory)	La memoria (RAM, buffers, caches) debe caber dentro de la capacidad disponible.	- RAM disponible: 256 MB - RAM requerida: ≤ 256 MB
6	 Comunicación (Bandwidth / Latency)	- El ancho de banda y la latencia de los buses / interconexiones deben ser suficientes. - Evitar cuellos de botella.	- Ancho de banda requerido: 2 GB/s - Ancho de banda disponible: ≥ 2 GB/s - Latencia máxima: $< X$ ciclos

Restricciones de diseño - Caso H.264 Decoder

- Dependencias de datos entre frames (30 FPS).
- Paralelismo disponible (68.4% del algoritmo)
- Número limitado de cores disponibles (4 cores @ 3.0 GHz).
- Sincronización segura de recursos compartidos.
- Reducir el tiempo total de simulación (32.59 s).

Trade-offs de diseño

TRADE-OFFS DE DISEÑO EN MPSoC / ESL (SCE)

Decisiones que implican sacrificar un objetivo para mejorar otro.

Elección A (Ventajas)

vs.

Elección B (Ventajas)

¿Qué se sacrifica?

1 Nivel de Abstracción vs. Precisión



ESL / TLM (Alta abstracción)

- Simulación rápida
- Exploración temprana
- Menor esfuerzo de modelado



RTL / Pin-Accurate (Baja abstracción)

- Máxima precisión temporal
- Ciclo a ciclo, pin a pin
- Detecta problemas de bajo nivel



• Precisión por velocidad
• Menos detalle temporal
• Visibilidad de señales de bajo nivel

2 Hardware vs. Software



Hardware (Aceleradores)

- Alto rendimiento
- Baja latencia
- Paralelismo masivo



Software (Procesadores)

• Flexibilidad
• Fácil actualización
• Menor costo de desarrollo

• Flexibilidad vs. rendimiento
• Mayor área y consumo (hardware)
• Menor velocidad (software)

3 Número de Procesadores vs. Complejidad



Más núcleos / Procesadores

- Mayor paralelismo
- Más rendimiento
- Menor tiempo de ejecución



Menos núcleos / Procesadores

• Menor complejidad
• Menor overhead de SO
• Menor consumo y área

• Rendimiento vs. complejidad
• Más núcleos generan overhead (sincronización, comunicación, SO)

4 Paralelismo vs. Overhead de Sincronización



Más paralelismo

- Menor tiempo de ejecución
- Mejor utilización de recursos



Menos paralelismo

• Menor overhead
• Menor contención
• Diseño más simple

• Tiempo de ejecución vs. overhead
• Más hilos → más locks, context switches y espera

5 Comunicación Compleja vs. Simplicidad



Red compleja (Crossbar, NoC)

- Mayor ancho de banda
- Menor latencia promedio
- Escalable



Red simple (Bus único)

• Menor área
• Menor consumo
• Diseño más simple

• Ancho de banda vs. área
• Más rendimiento vs. simplicidad / costo

Tradeoffs de diseño - Caso H.264 Decoder

- **Paralelismo vs Overhead de sincronización.**
- **Complejidad arquitectónica vs Rendimiento (Tiempo: 32.59 s vs 17.76 s, Utilización CPU: 99% vs 197%).**
- **Speedup ideal vs Costos reales de implementación (2.05× ideal, 1.84× real).**

Herramientas y metodologías de exploración de diseño utilizadas

Metodología specify-explore-refine (SER)

Toda la exploración de diseño en SCE se basa en una metodología: **specify-explore-refine**. Es un flujo **top-down dirigido por el diseñador**.

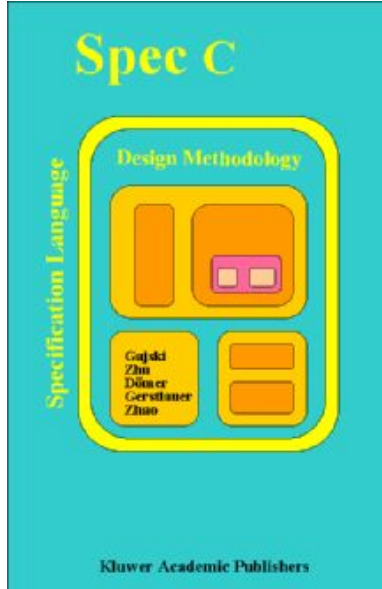
Las tres fases:

- **Specify** — se parte de un modelo abstracto que describe *qué* hace el sistema (sin arquitectura todavía).
- **Explore** — el diseñador toma decisiones de diseño explorando el espacio de soluciones (vía GUI o scripting).
- **Refine** — la herramienta genera automáticamente un modelo nuevo, en un nivel de abstracción más bajo, integrando esa decisión.

Etapas de Exploración

#	Etapas	Función	Modelo de Salida
1	Architecture Exploration	PE Allocation · Beh/Var/Ch Partitioning	Modelo de Arquitectura
2	Scheduling Exploration	Static Scheduling · OS Task Scheduling	Scheduled Model
3	Network Exploration	Bus Network Allocation · Channel Mapping	Network model
4	Communication Synthesis	Bus Addressing · Bus Synchronization	Communication model (que es un TLM , o alternativamente un PAM — Pin-Accurate Model)

SpecC



SpecC es el **lenguaje de descripción a nivel de sistema (SLDL)** sobre el que opera todo el flujo. Es una **extensión de C**.

A diferencia del código C/C++ plano y secuencial, SpecC expresa **explícitamente** en el modelo:

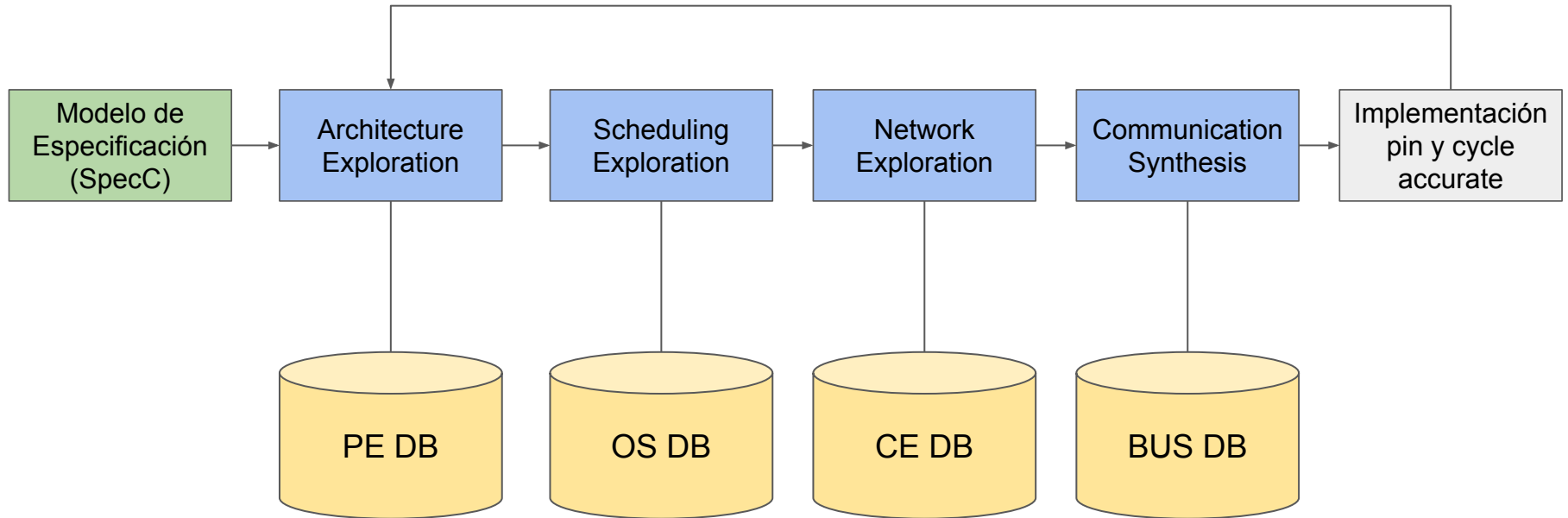
- Jerarquía conductual y estructural
- Paralelismo y pipelining (constructos `par{}` y `pipe{}`)
- Comunicación separada en canales (channels)
- Restricciones de tiempo (timing)

Fuente: *The SpecC System*, Center for Embedded Computer Systems, University of California, Irvine.
Disponible en:
<https://www.cecs.uci.edu/~specc/>

Otras Herramientas y Bases de Datos

- **Herramientas de refinamiento** (architecture / scheduling / network) — generan cada modelo automáticamente
- **GUI / Scripting** — interfaz por la que el diseñador decide
- **Library Builder** — mantiene las bases de datos (agregar/quitar/modificar componentes)
- **RTL Synthesis y SW Synthesis** — generan el Verilog y el binario finales
- **Simulador DE** — valida cada modelo (extendido a multi-core: la contribución del paper)
- **PE DB** — procesadores y aceleradores
- **OS DB** — modelos de RTOS
- **CE DB** — elementos de comunicación (puentes, transducers)
- **Bus DB** — protocolos de bus
- **RTL DB / SW DB** — para la síntesis final

Flujo completo de exploración



Aplicaciones prácticas y relevancia actual del trabajo presentado

Aplicaciones prácticas:

- Diseño temprano de MPSoCs heterogéneos.
- Prototipado virtual.
- Validación más rápida.

Relevancia actual:

- Los sistemas actuales todavía son heterogéneos y complejos, por lo que el enfoque ESL/top down sigue siendo útil
- Hardware multi-core es sigue siendo estándar, por lo que el paralelismo sigue siendo una necesidad real.
- La herramienta es del 2010 pero la metodología sigue vigente.

Conclusiones

Ventajas

- **Abstracción y diseño:** Ofrece una metodología *top-down* que facilita explorar alternativas.
- **Simulación paralela:** Reduce drásticamente el tiempo de validación usando multi-core.
- **Escalabilidad en MPSoC:** Ideal para manejar la complejidad de sistemas embebidos grandes.

Desventajas o limitaciones

- **Dependencia del modelo:** El éxito depende estrictamente del paralelismo disponible en la aplicación.
- **Sobrecosto por sincronización:** El *speedup* real es menor al teórico y no escala perfectamente.
- **Complejidad técnica:** Aumenta la complejidad del simulador y existe riesgo de subutilizar núcleos.

Referencias

- W. Chen, X. Han, and R. Dömer, "ESL design and multi-core validation using the System-on-Chip Environment," in *Proc. IEEE Int. High Level Design Validation and Test Workshop (HLDVT)*, Anaheim, CA, USA, Jun. 2010.
- Metodología SER: A. Gerstlauer, J. Peng, D. Shin, D. Gajski, A. Nakamura, D. Araki, and Y. Nishihara, "Specify-explore-refine (SER): From specification to implementation," in *Proc. 45th Annu. Design Automation Conf. (DAC '08)*, Anaheim, CA, USA, Jun. 2008, pp. 586–591, doi: 10.1145/1391469.1391617.
- Center for Embedded Computer Systems, "The SpecC System," University of California, Irvine. [Online]. Disponible: <https://www.cecs.uci.edu/~specc/>. [Consultado: 11-jun-2026].

Declaración de Uso de Inteligencia Artificial

Modelo	Usos	Prompt
Claude (Anthropic), modelo Opus 4.8	Síntesis y organización del contenido de la exposición, consulta de conceptos técnicos del paper, localización de información exacta en el artículo, diseño de un diagrama de apoyo, estructuración de las diapositivas y corrección de redacción y formato de citas.	"Apóyame en la preparación de una exposición sobre el paper, diseñando un diagrama de apoyo, estructurando las diapositivas y corrigiendo la redacción y el formato de las citas."